

PRML PROJECT PRESENTATION

Predictive Maintenance

GROUP 25

Arnav Vivek (B24CS1014)
Hayagriv Desikan (B24CS1092)
Anjaneya B Damle (B24EE1004)
Pranav S Kala (B24EE1058)

Problem Statement : Predictive Maintenance

The Need for Predictive Maintenance

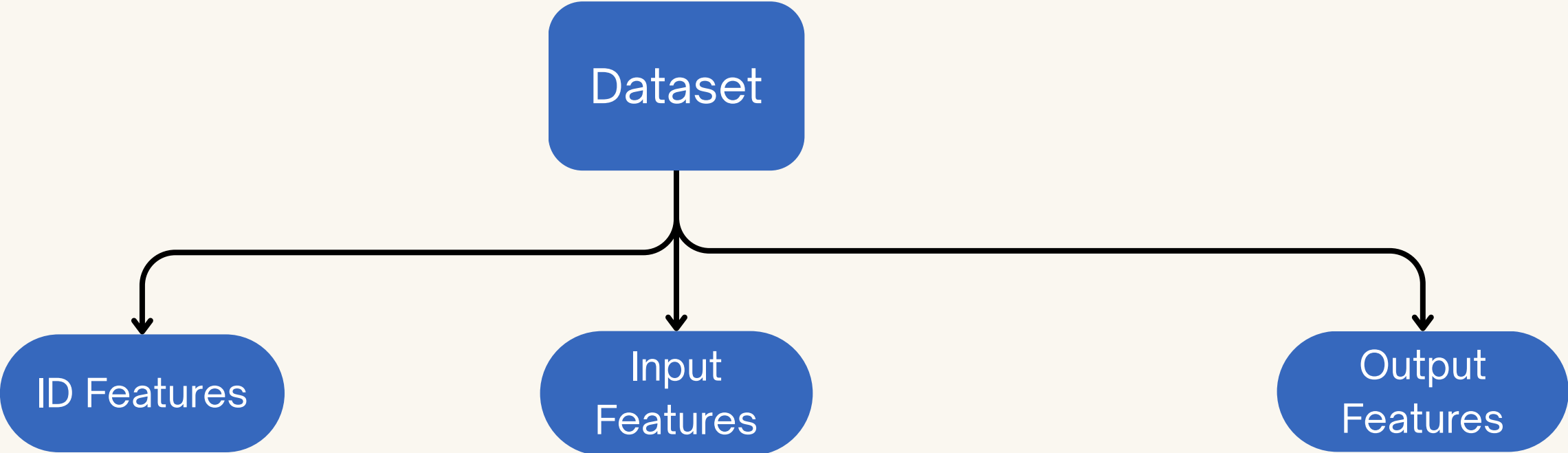
- Industrial machines generate continuous sensor data:
 - Temperature, rotational speed, torque
- Traditional maintenance approaches:
 - Reactive → Fix after failure
 - Preventive → Fixed schedule (even if not needed)
- Uses real-time sensor data to:
 - Detect patterns and anomalies
 - Identify early signs of failure
- Machine Learning Model:
 - Learns from historical data
 - Predicts Machine Failure (Yes/No)
 - Predicts type of failure

Problem Statement : Predictive Maintenance

Predictive Maintenance using Machine Learning

- | | |
|---|--|
| <ul style="list-style-type: none">• Inputs:<ul style="list-style-type: none">◦ Type◦ Air Temperature◦ Process Temperature◦ Rotational Speed◦ Torque◦ Tool Wear | <ul style="list-style-type: none">• Output:<ul style="list-style-type: none">◦ Failure prediction before breakdown• Benefits:<ul style="list-style-type: none">◦ Reduced downtime◦ Cost-effective maintenance◦ Increased machine lifespan◦ Improved operational efficiency |
|---|--|

Dataset Description



Highly imbalanced dataset
Machine failures are very rare

Ratio of 0:1 is 28:1

UID	Product ID
1	M14860
2	L47181
3	L47182
4	L47183
5	L47184

Type	Air temperature	Process temperature	Rotational speed	Torque	Tool wear
M	298.1	308.6	1551	42.8	0
L	298.2	308.7	1408	46.3	3
L	298.1	308.5	1498	49.4	5
L	298.2	308.6	1433	39.5	7
L	298.2	308.7	1408	40.0	9

Machine failure	TWF	HDF	PWF	OSF	RNF
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Models Used

1
Balanced class Logistic
Regression

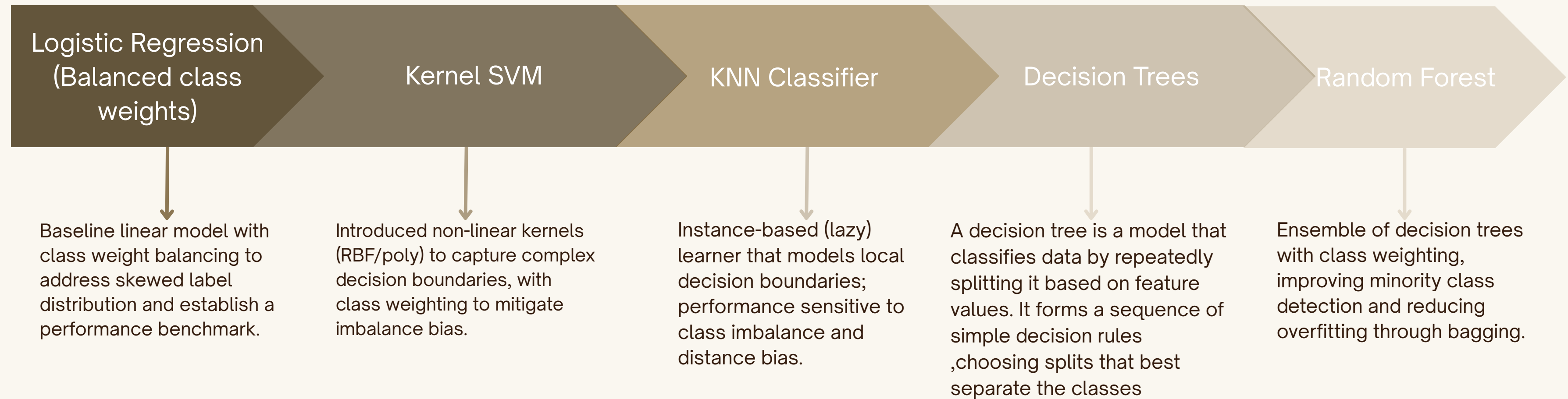
2
Kernel SVM

5
Random Forest

3
KNN

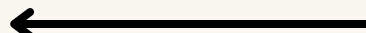
4
Decision Tree

Model Evolution



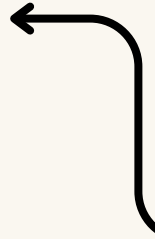
Preprocessing (Common to all models)

Dropping ID features (Not relevant features in the context of maintenance of machine)



UID	Product ID
1	M14860
2	L47181

Scaling numerical features: as the magnitudes of different sensor readings were of different orders



```
preprocessor = ColumnTransformer(transformers=[  
    ('num', StandardScaler(), num_features),  
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)  
)
```

One hot encoding of the one categorical feature [Type: L M H] to three numerical binary valued features



Dataset Splitting

Dataset has an 80/20 Train Test Split.



Ensure no data leakage between train and test sets

Same split (Random state 42) used across all models for fairness and comparability

Hyperparameter Tuning

Model	Hyperparameter Tuned	Metric used
Logistic regression	Regularization Strength	F1 Score
KNN	K values	F1 Score
Kernel SVM	Choice of kernel	F1 Score
Decision Trees	Threshold	F2 score
Random Forest	Threshold	F2 score

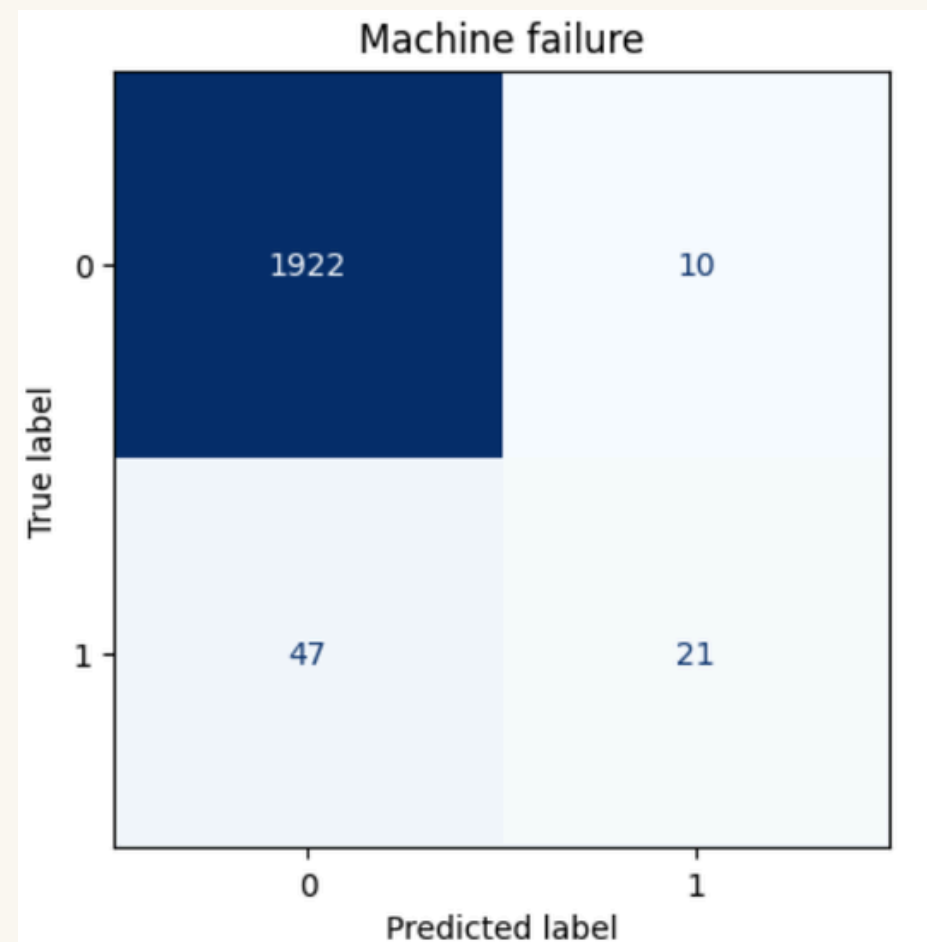
Evaluation Metrics

- Accuracy is misleading under severe class imbalance, as it inflates performance by favoring the majority class.
- The objective prioritizes minimizing false negatives over false positives due to the asymmetric cost of missing rare but critical failure events.

Metric	Purpose
Precision	Control false positives (important when false alarms are costly)
Recall	Capture as many true positives as possible (important when missing a failure is costly)
F1 Score	Balance between precision & recall (general performance metric)
F2 Score	Prioritize recall more than precision (used when missing positives is very bad)

KNN and Logistic Regression

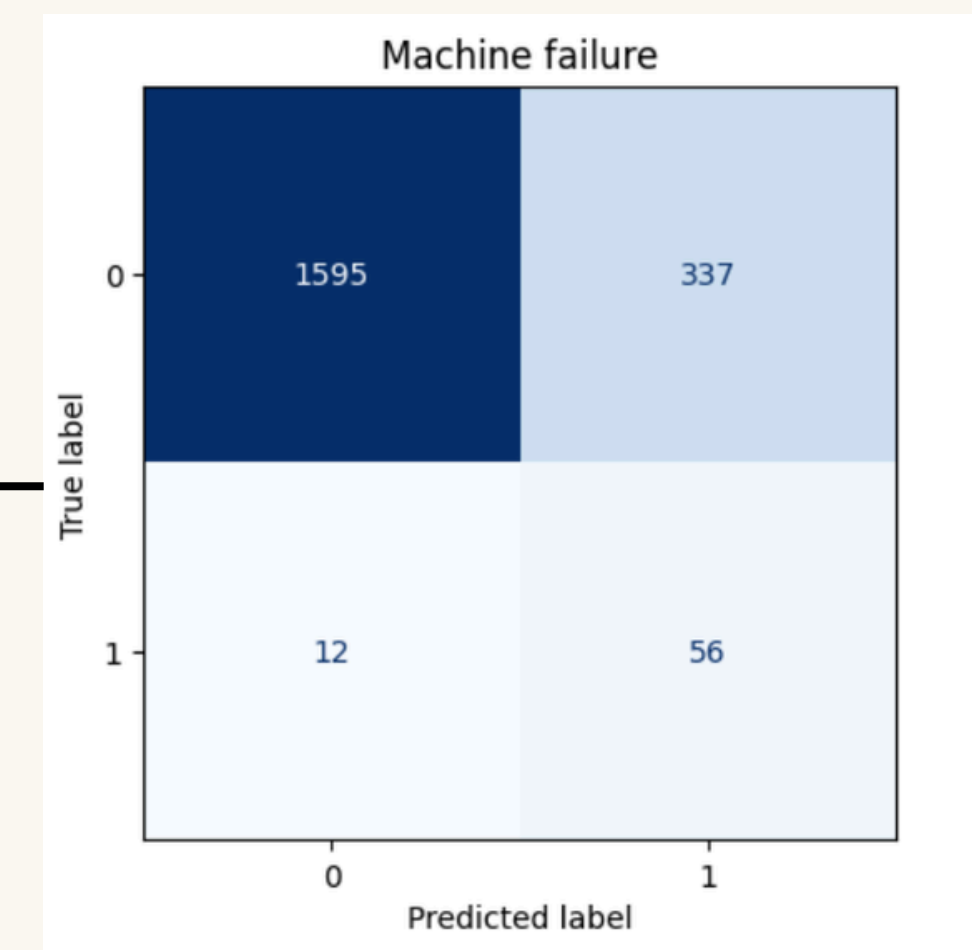
KNN



KNN fails due to severe class imbalance causing neighborhoods to be dominated by normal samples and misclassifying failures. Additionally, failures are sparse and not clustered, so distance does not reflect true similarity, making Euclidean distance ineffective.

Logistic regression fails because the dataset is highly imbalanced and the failure label arises from multiple distinct mechanisms, leading to a non-linear and localized decision boundary that cannot be captured by a single linear discriminant function.

LOGISTIC REGRESSION



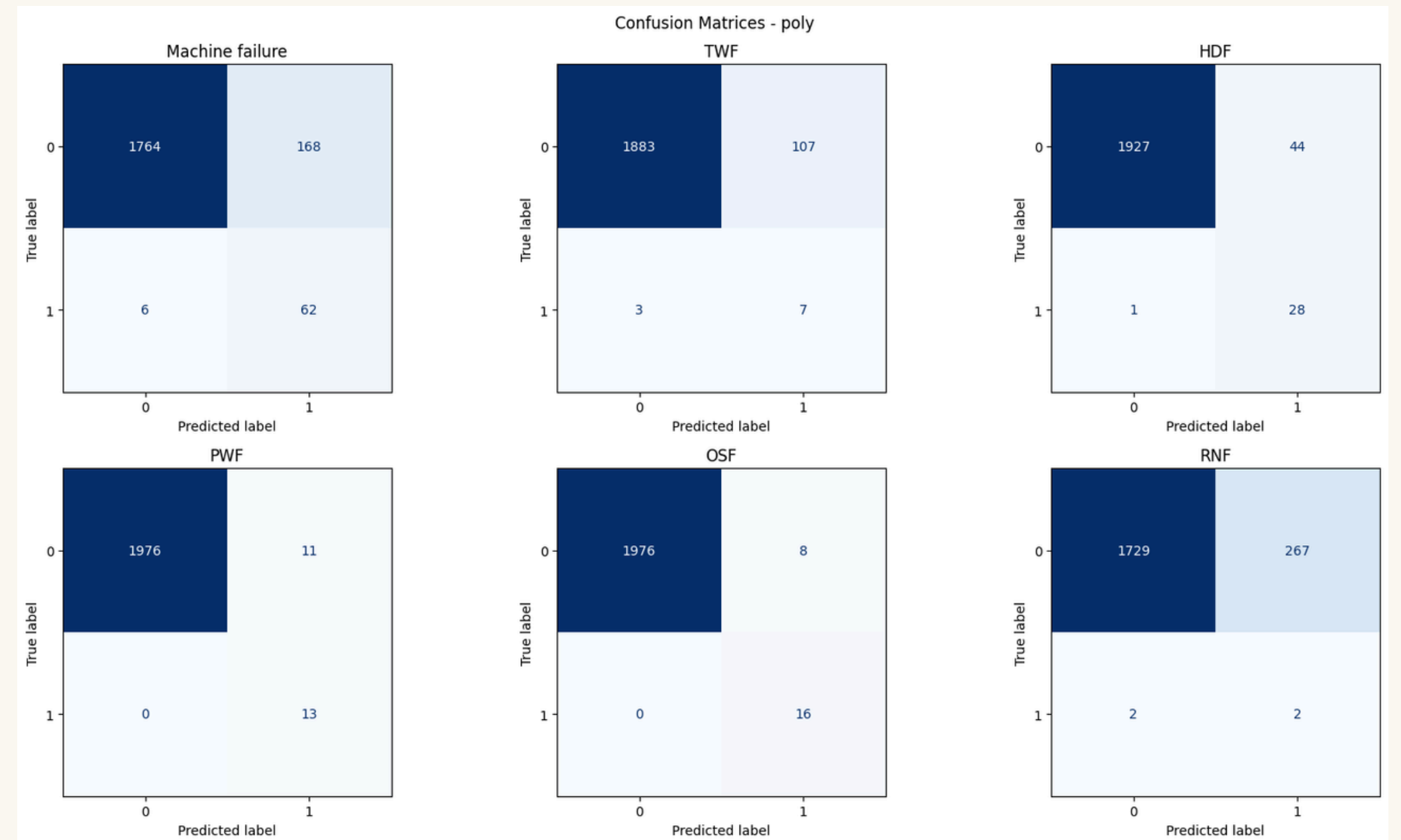
The K-Nearest Neighbors (KNN) model performs poorly, as it produces a high number of false positives while still maintaining a relatively low number of false negatives.

Logistic Regression achieves a reasonable number of true positives; however, it also produces a high number of false positives, which must be reduced alongside false negatives to improve overall model performance.

Kernel SVM

- Strong performance with very few false negatives
- High rate of false positives remains a challenge
- Catching failures is important, but too many false alarms raise unnecessary maintenance costs
- Very few missed failures
- Too many false positives
- Extra alerts can lead to unnecessary spending on maintenance
- Reliable at spotting real failures
- Struggles with many incorrect alerts
- Reducing false alarms helps control maintenance costs

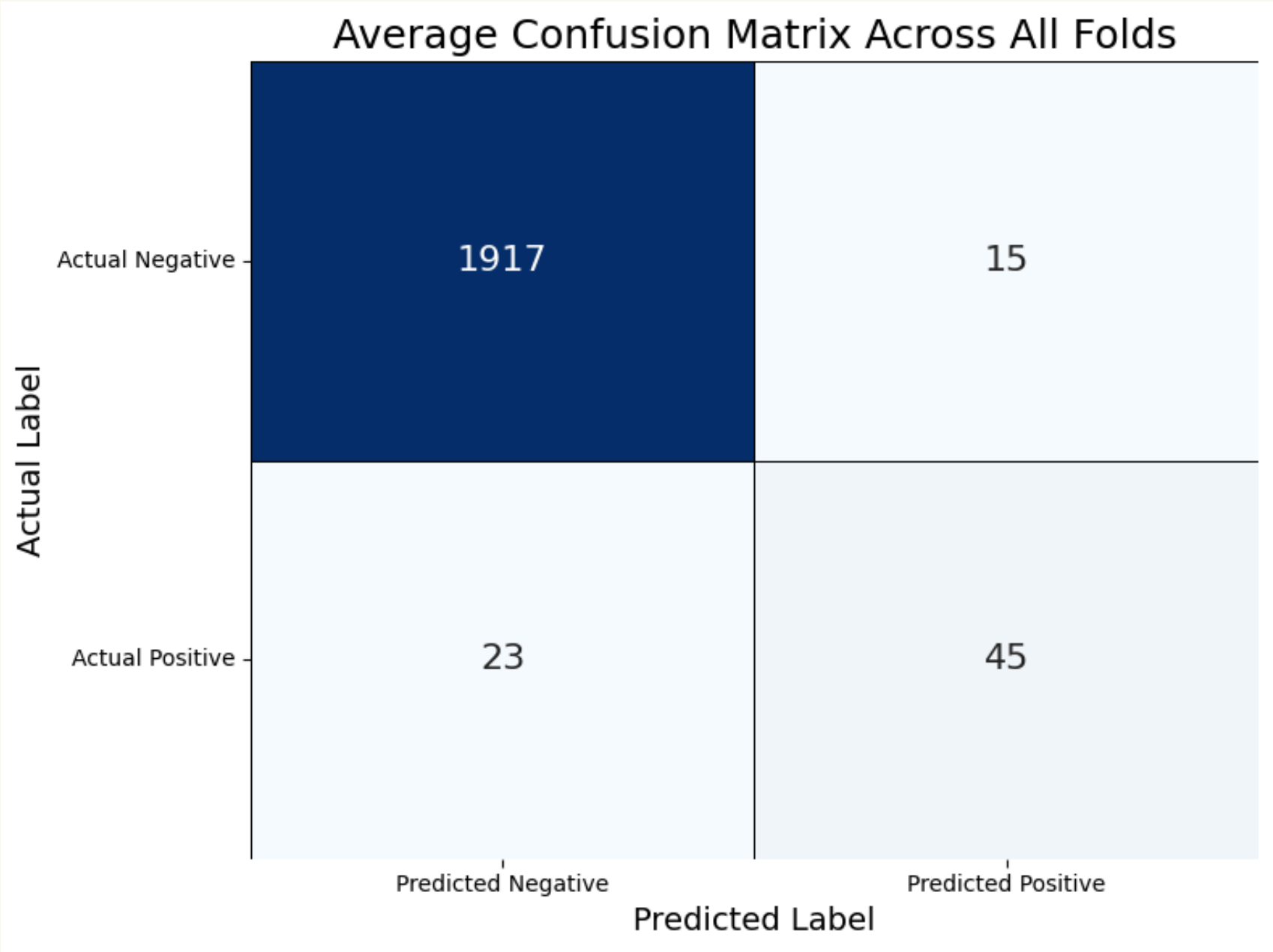
Kernel SVM struggles with imbalanced data because it treats both classes equally, so the majority class dominates the decision boundary—leading to poor minority detection or excessive false alarms when tuned for recall.



Decision Trees

Decision trees were chosen because they can model non-linear decision boundaries and capture rule-based structures, allowing representation of multiple failure modes as combinations of conditions. By partitioning the feature space into regions that isolate failure cases, they improve detection of the minority class and reduce false negatives. They also demonstrated superior empirical performance, validated through 5-fold cross-validation.

The decision tree achieved an average F2-score of 0.681 ± 0.055 across 5-fold cross-validation, indicating stable performance with strong recall for failures. The average depth (~ 12) and pruning ($\alpha \approx 0.00025$) balance complexity and generalization. It reduces false negatives effectively, though some variability exists due to sparse failure cases.

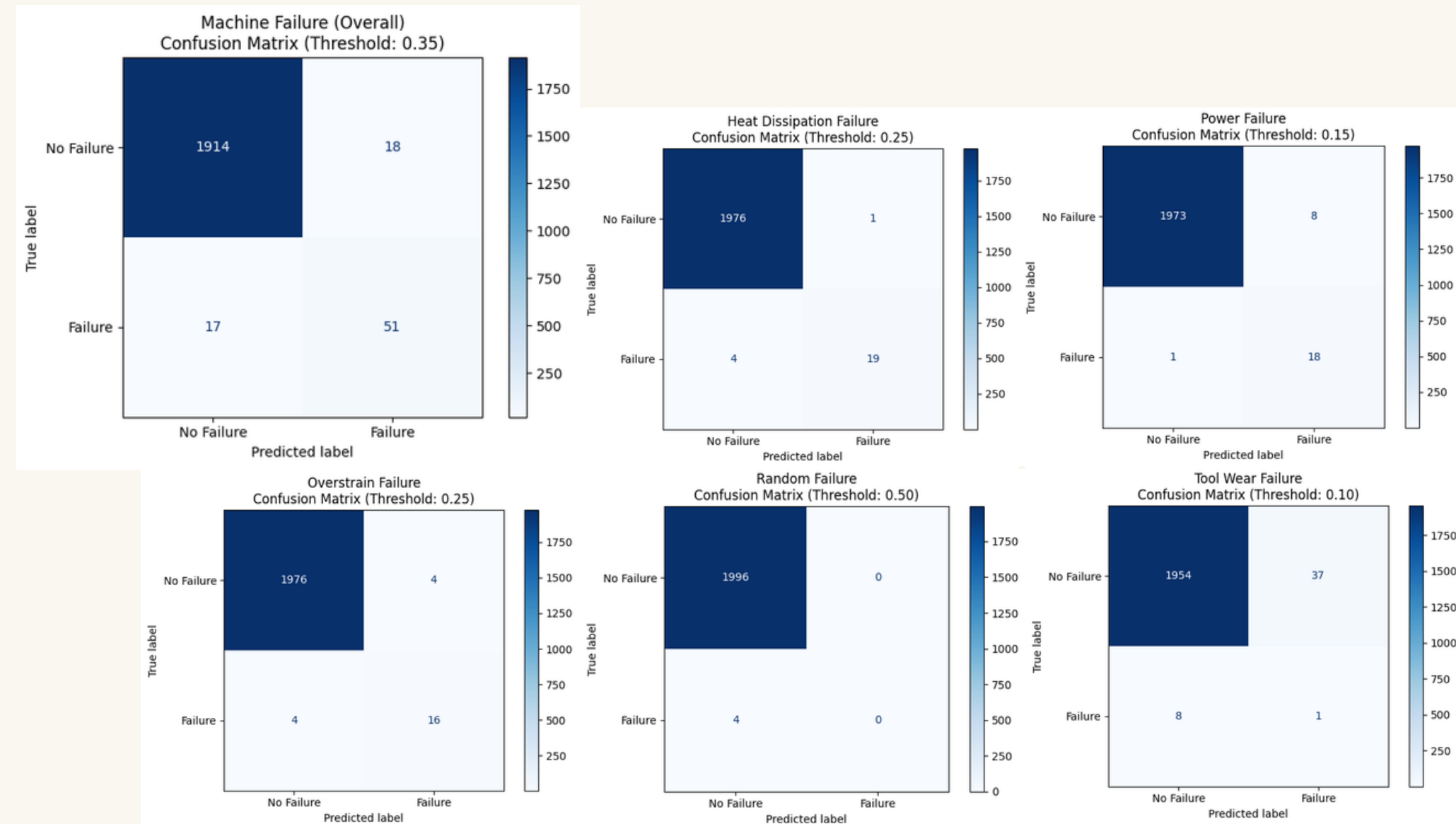
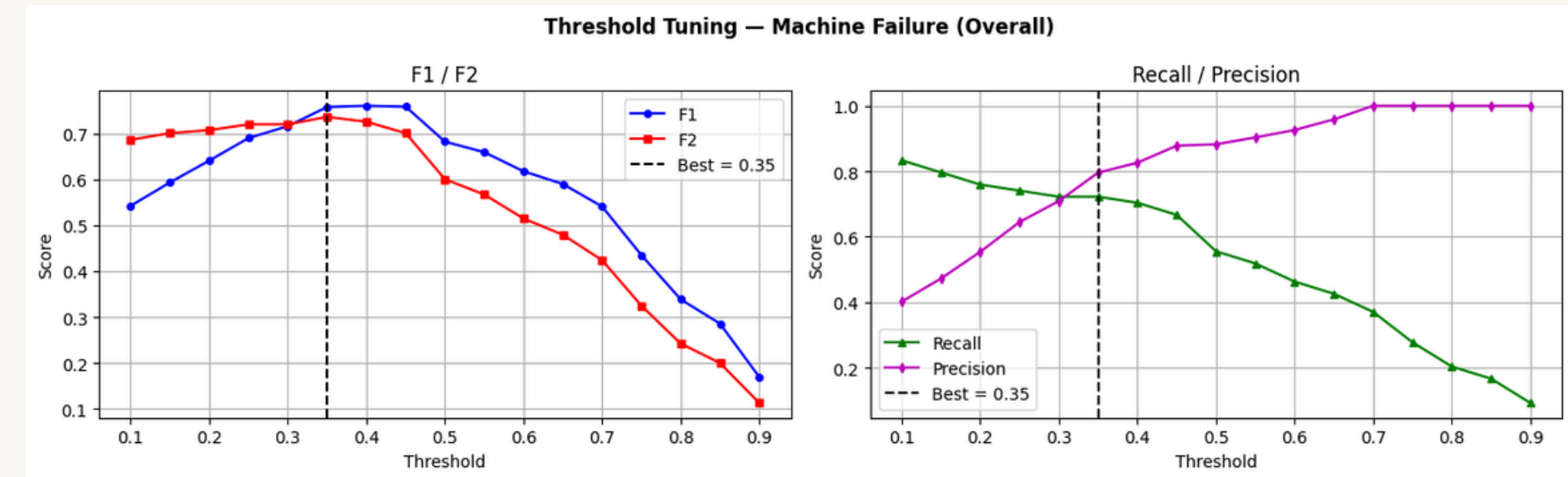


Random Forest

Random Forest enhances the performance of a single decision tree by minimizing variance through the averaging of multiple trees, each trained on different subsets of the data.

- This results in improved generalization and offers a more reliable approximation of the decision boundary.
- As failures in the dataset occur in various distinct regions, different trees identify different failure patterns.
- The aggregation of these trees enhances the detection of the minority class, thereby reducing false negatives.

This aligns with the fundamental principle that combining several classifiers typically leads to better performance than relying on a single model.



Thank you Work division:

Arnav Vivek (B24CS1014): Random forest
Hayagriv Desikan (B24CS1092): Decision Trees
Anjaneya Damle (B24EE1004): Kernel SVM
Pranav Kala (B24EE1058): Logistic Regression, KNN

Thanks to Mamkesh Sir, our project mentor for discussing our approach and suggestions on the model